

Roteiro Dano e Invencibilidade Temporária

Introdução

Eaí, galerinha! Beleza?! Sejam bem vindos de volta a mais uma aulinha do CodeQuest. Meu nome é [nome do apresentador], e hoje, a gente vai aprender a como fazer o seu personagem tomar dano, e também a fazer aquele sisteminha de invencibilidade temporária para impedir que ele seja combado pelo inimigo.

Detecção de Dano

(Estou supondo que já foi explicado o sistema de vida. Caso não, dá pra utilizar isso daqui)

Muito bem, então. Antes de tudo, precisamos definir a vida do nosso personagem. Afinal, se ele não tiver vida, ele não pode tomar dano. Então, para isso, vamos abrir o código do nosso player, e criar duas variáveis: vida máxima e vida atual.

Nesse exemplo, a vida máxima do nosso personagem será fixa. Ou seja, não é algo que vamos modificar. Então podemos definir ela como uma constante.

[criar a const MAX_HEALTH = 100]

Beleza. E agora, criamos a nossa vida atual. Como a vida atual é algo que pode ser modificado conforme jogamos o jogo, ela vai ser uma variável normal.

[criar a var current_health]

Ok, e por fim. Como, no início de toda fase, a vida do personagem sempre vai estar cheia, podemos adicionar a seguinte linha na nossa função *ready()*:

[escrever current_health = MAX_HEALTH]

Pronto. Agora sim, vamos fazer o nosso personagem tomar dano, de fato.

(Daí agora, se não tiver que explicar a vida, coloca a introdução aqui)

Bem, para fazer o nosso personagem tomar dano, nós vamos abrir o código do nosso player, e criar uma função para isso.

```
[criar a função func take_damage(damage: int)]
```

Ok, e agora, nessa função, a gente simplesmente coloca o seguinte comando:

```
[escrever current_health -= damage]
```

Bem simples, não?

Daí agora, é só a gente ativar essa função no objeto. Então, por exemplo, vamos fazer os projéteis dos inimigos dar dano.

A gente abre aqui o objeto dos projéteis inimigos. E criamos a seguinte função: `_func_on_body_entered(body)`:

```
[cria a função]
```

E atribuímos que quando o jogador entra em contato com o projétil, ele toma, por exemplo, 10 de dano.

```
[Escreve:
```

```
if "Player" in body.name:  
    body.take_damage(10)]
```

E pronto. Agora, quando o nosso personagem toma o projétil, ele vai tomar 10 pontos de dano.

Controle de Tempo

Legal, mas... do jeito que está, a gente ainda tem um problema. Do jeito que está agora, o nosso personagem vai ser combado pelos inimigos. Então, os 10 de dano que ele tomaria, viraria, por exemplo, 30 de dano. Para evitar isso, a gente precisa deixar o nosso personagem invencível, pelo menos durante um pequeno tempo, até ele voltar a receber dano.

Para fazer isso, nós iremos precisar de uma lógica de timer. Existem duas formas de fazer isso: utilizando o componente de timer; e colocando um timer próprio na função de tomar dano.

(Essa parte aqui é bem parecida com a aula 10, que está no YouTube. Acho que uma referência à ela pode ser válida. Se essa aula for uma das mais avançadas, inclusive, acho que nem precisa explicar, apenas referenciar.)

Na primeira forma, a gente precisa colocar um nóculo de timer no script do player, e quando a função rodar, a gente coloca a variável `can_take_damage` como verdadeiro.

[escreve tudo isso no código]

Aí quando o jogador tomar dano, é só iniciar o timer.

[escreve o resto do código]

Na segunda forma, a gente não utiliza o nóculo do timer, mas em vez disso, a gente coloca a lógica do timer dentro da função de tomar dano.

Dessa forma, a gente inicia a função de tomar dano com a variável `can_take_damage` como sendo falsa, e daí, no final da função, a gente coloca o comando `await`, junto com o tempo de invencibilidade que a gente quer. Por fim, depois desse comando, a gente coloca a variável como verdadeira de novo.

Ambos os métodos têm o mesmo resultado, no final: depois que o personagem tomar um dano, ele não pode mais tomar dano durante um determinado tempo. Nesse meio tempo ele é invencível.

Feedback Visual

Muito bem. Agora o nosso personagem já toma dano, e depois ele fica invencível durante alguns segundos. Falta mais alguma coisa? Bem, aí entram os detalhes, que são importantes no desenvolvimento de jogos, e que vão melhorar a qualidade do seu produto final.

Nessa aula, o detalhe que a gente vai adicionar, vai ser um indicador visual, para indicar que o nosso personagem tomou dano. O indicador vai ser algo bem básico: o personagem pisca em vermelho.

Bom, vamos lá. No nosso script do personagem, a gente vai na função `take_damage` de novo. Depois que o hp dele diminui, a gente vem, e coloca um comando que vai fazer o sprite dele ter uma coloração vermelha.

```
[escrever o sprite.modulate = Color(1, 0.3, 0.3)]
```

Esse comando, basicamente, troca a coloração do sprite do personagem para uma vermelha. Porém, do jeito que está, ele vai deixar o nosso bonequinho vermelho para sempre. Então, em baixo dele, a gente coloca o comando para fazer ele voltar de novo para a coloração normal.

```
[escrever o sprite.modulate = Color(1, 1, 1)]
```

Porém, como vocês já devem saber, esses comandos são feitos em nanossegundos. Então você não vai conseguir ver nada, se o comando estiver assim. A gente tem que fazer com que ele fique um tempinho, colorido em vermelho, antes de voltar para branco...

E bem, se você chegou até aqui, você provavelmente já sabe a resposta: timers.

```
[escreve o comando do await entre os dois sprite.modulate]
```

E bem, agora sim, tudo deve estar funcionando.

Finalização

```
[mostra o jogo sendo testado, com as novas funcionalidades]
```

E bem, pessoal. Como podem ver, o nosso personagem agora é capaz de tomar dano.

Bom, pessoal. Pra aula de hoje é isso. Como desafio, fica para vocês depois tentarem fazer um sistema de cura.

Nos vemos então, na próxima aula do CodeQuest! Falous!!!